Logic Programming

Rodica Potolea
Camelia Lemnaru
Richard Ardelean

Lecture #11

Graph traversal

TECHNICAL UNIVERSITY OF CLUJ-NAPOCA

Agenda

- **Built-in predicates (and relationship to graphs)**
  - *findall* and more
- **Constructs and relationship with graphs**
  - For all is true
  - Application – same graph
- **Graph Traversal**
  - DFS
  - BFS

Built-in predicates
their utility in graph searching

- `findall` – selects all items (and only those items) with a certain property from a knowledge-base
- You must know the predicate to use it appropriately
  - Know HOW it is implemented
    - to utilize the strategy in graph problems

findall

- (example from Clocksin and Mellish)
- Suppose we have a knowledge base of some drinks lovers:

```
% likes/2 (+Person, +Drink).

likes(bill,  wine).
likes(dick,  beer).
likes(harry, beer).
likes(john,  beer).
likes(peter, wine).
likes(tom,   beer).

[q] ?- findall(X, likes(X,beer), L).
true, L=[dick,harry,john,tom].
```

findall – implementation

```
% findall/3 (+CollectedVar, +CollectorPred, -CollectorVar).

findall(X,P,_):-
        P,

        asserta(found(X)),
        fail.
findall(_,_,L):-
        collect_found([],L).

collect_found(P, L):-
        retract(found(X)), !,

        collect_found([X|P],L).
collect_found(L,L).
```

% uses an auxiliary knowledge base (**akb**) - found
% = call(P), P is a predicate;
% its exe instantiates some argument X.
% puts P's instantiated argument on top of **akb**
% request for backtrack to P.
% starts collecting from the **akb**,
% initializes the partial result to empty list.

% extracts from top of the **akb**
% succeeds if a genuine data; fails when no more
% adds it in front and go recurse

```
likes(bill,  wine).
likes(dick,  beer).
likes(harry, beer).
likes(john,  beer).
likes(peter, wine).
likes(tom,   beer).
```

```
[q] ?- findall(X, likes(X,beer), L).
true, L=[dick,harry,john,tom].
```

findall – implementation

```
% findall/3 (+CollectedVar, +CollectorPred, -CollectorVar).

findall(X,P,_):-
        P,
        asserta(found(X)),
        fail.
findall(_,_,L):-
        collect([],L).

collect_found(P,L):-
        retract(found(X)), !,
        collect_found([X|P],L).
collect_found(L,L).

% What type of recursion is used in collect?
```

findall – execution

```
findall(X,P,_):-
        P,
        asserta(found(X)),
        fail.
findall(_,_,L):-
        collect_found([],L).

collect_found(P,L):-
        retract(found(X)), !,
        collect_found([X|P],L).
collect_found(L,L).
```

% initial kb

```
likes(bill,  wine).
likes(dick,  beer).
likes(harry, beer).
likes(john,  beer).
likes(peter, wine).
likes(tom,   beer).
```

% akb

```
found(tom).
found(john).
found(harry).
found(dick).
```

%created bottom-up ↑ collected top-down ↓

[q] ?- findall(X, likes(X,beer), L).
true, L=[dick,harry,john,tom].

findall

- General call:

```
?- findall(X,P,L)
```

  - where `X` is a variable
  - `P` a predicate and `X` occurs in `P`
  - `L` would collect all `X`'s so that `P`'s execution succeeds on `X`
- Creates the list `L` of `X`'s that satisfy `P`, in the order of occurrence in the original knowledge base

`findall` – related

- Associate predicates:

`setof(X,P,S)`

  - Creates the set of terms (standard order, without duplicates; represented also as list) of `X`'s that satisfy `P`. If none, fails.
  - **Differences to** `findall`:
    - Order: standard vs as found in the knowledge base
    - Duplicates: no vs found in the knowledge base
    - No term: fails vs empty list

`bagof(X,P,S)`

  - Same as `setof` BUT list NOT ordered + may have duplicates
  - So same as `findall` but fails if no term matches

```
findall(X,P,L):- bagof(X,P,L),!.
findall(_,_,[]).
```

[q] ?- for_all_is_true(member(Char, ['a','b','c']), char_type(Char, lower)).
[q] ?- for_all_is_true(member(Char, ['a','B','c']), char_type(Char, lower)).

For all is true technique

- Construct to check if `for_all Predicate1 is_true Predicate2`
- Aims to identify/verify whether in all cases when P1 is true, P2 is true as well. Thus, checks P1=>P2.
- Template looks like:

```
for_all_is_true(X,Y):-
      X,                    % calls X, a predicate,
                            %       whose execution may instantiate hidden args
                            % calls Y in the same context. If succeeds, its negation fails,
      not(Y),!,             %       and backtracks to X who is executed again, with a
                            %       new instance of args; generates a new context
                            %       the first context of X where Y fails, succeeds, and !
                            %       is irreversible crossed and the predicate fails
      fail.                 %       overall.

                            % if we reached here, in all contexts when X holds true
for_all_is_true(_,_).       % succeeds as well, therefore, X=>Y
```

For all is true in graphs context

- Given graphs G1 and G2 with neighbor and edge representations respectively, do they represent the same graph?

G1

```
neighbor(Node,List_of_Neighbors)
```

G2

```
edge(Node1,Node2)
```

- G1 and G2 are the same means G1⇔G2 which is (G1=>G2)+(G2=>G1)

(G1=>G2)

```
for_all edges_in_G1
is_true edge_in_G2
```

I

(G2=>G1)

```
for_all edges_in_G2
s_true edge_in_G1
```

- Define the edges in the 2 representations:

An edge in G1: (defines the nondeterministic check for all edges)

```
is_edge_1(X,Y):-
        neighbor(X,L),
        member(Y,L).
```

% to find all edges in G1, call nondeterminist(X,Y,free)

% finds first pair first (and next on backtrack), instantiates both X and L.

% nondeterm call on member; instantiate Y, one at a time, eventually all.

An edge in G2:

```
is_edge_2(X,Y):-
        edge(X,Y);
        edge(Y,X).
```

Same graph?

- (G1=>G2)

```
for_all edges_in_G1
is_true edge_in_G2
```

(G2=>G1)

```
for_all edges_in_G2
is_true edge_in_G1
```

- Denote the `for_all_is_true` as `eq1` and `eq2` depending on the implication (G1=>G2) respectively (G2=>G1) we verify

```
eq1:-
        is_edge_1(X,Y),          % for each edge in the first representation
        not(is_edge_2(X,Y)),!,   % there is one edge in the other graph
        fail.                    % otherwise, we reach here, hence, fail.
eq1.                             % if we get here, G1=>G2 shown

eq2:-
        is_edge_2(X,Y),
        not(is_edge_1(X,Y)),!,
        fail.
eq2.

same_graph:- eq1, eq2.
```

```
edge(a,b).
edge(b,a).
neighbor(a,[b]).
neighbor(b,[a]).

[q] ?- same_graph.
```

Change representation of a graph

- Given graph **G1** a weighted, edge representation and we need to get **G2** with neighbor representation

G1

```
edge(a,b,15).
```

G2

```
neighbor(a,[p(b,15),…]).
```

```
edge2neighb:-
        is_edge(X,_,_),
        not(neighbor(X,L)),
        findall(Z,convert(X,Z),L),
        assertz(neighbor(X,L)),
        fail.
edge2neighb.

convert(X,Z):-
        is_edge(X,Y,W),
        Z=p(Y,W).
```

- Note:

`findall(Z,succ(X,Z),L)` same as `findall(p(Y,W),is_edge(X,Y,W),L).`

- Given G2, generate G1. Homework.

```
collect([X|R]):-
        retract(node(X)),!,
        collect(R).
collect([]).
```

Depth first search

- Assume undirected, unweighted graph, edge representation
- 2 stage process:
  1. 1st clause: make the trail placing in a queue the vertices in order of visitation (being in the additional knowledge base `node` - hence a dynamic predicate - is as if we have a grey vertex).
  2. 2nd clause: when done, collect them.

```
dfs(X,L):-
        assertz(node(X)),       % place start/current node as visited
        is_edge(X,Y),           % take first/next neighbor of current node X
        not(node(Y)),           % if already visited, backtrack to take another;
        dfs(Y,L).               %       if not, continue from Y
dfs(_,L):-
        collect(L).             % similar to collect in findall, but on node akb.
```

- It is covering just the connected component of the starting vertex (is similar to dfs_visit in Cormen).
  - Need a wrapper to re-run it from all connected components (all remaining white nodes – color NOT implemented here).

TECHNICAL UNIVERSITY OF CLUJ-NAPOCA

Breadth first search

```
collect([X|R]):-
        retract(node(X)),!,
        collect(R).
collect([]).
```

- Assume undirected, unweighted graph, edge representation
- Queue made by the akb named `node` (hence dynamic predicate)

```
bfs(X,_):-
        assertz(node(X)),       % add at the end of the Q the current node
        node(Y),                % reads from front of Q, gets Y instantiated (initially X=ONLY one in Q)
        is_edge(Y,Z),           % take first/next neighbor of current Y (gets Z instantiated);
                                %       if none, backtrack and take next from Q
        not(node(Z)),           % if Z already visited, backtrack to take another;
        assertz(node(Z)),       % if not, put Z in Q and
        fail.                   % backtrack anyway to another neighbor of Y
bfs(_,L):-
        collect(L).             % similar to collect in findall, but on node in akb.
```

- There is an issue with this implementation for some languages/dialects (due to an optimization strategy). Oral explanation.

BFS – queues explicit

- BFS with 3 arguments representing:
  - Queue (`Q`) = list of candidates = list of items we reached, yet not finalized to process = grey nodes. Implemented as incomplete lists. Why? Try to find out. Oral discussion!
  - Expanded list (`Exp`) = list of items reached and processed = black nodes. Implemented as regular (complete) lists.
  - Result (`R`) = nodes in order of bfs processing. Copy of Expanded list when processing ends.

```
% bfs /3 (+Queue, +ExpansionList, -Result).

[q] ?- bfs([a|_], [], R).

bfs(Q,R,R):- var(Q),!.                  % when Q is empty, end exe, the Exp becomes R
bfs([X|Q],Exp,R):-                      % else, take first from Q
        expand(X,Q,Exp),                % and expand (put all white neighbors in Q) and
        bfs(Q,[X|Exp],R).               % continue by moving it in expanded
                                        % = make it black
```

BFS – queues explicit - contd

- The expansion step (2-stage process) – decides which neighbors to add in Q
- dynamic predicate `desc`; potential neighbors stored in auxiliary knowledge base (akb)

```
expand(X,_,Exp):-
        is_edge(X,Y),          % nondeterministically take Z, first neighbor of X
                               % (eventually all of them)
        not(member(Y,Exp)),    % should NOT be already processed (not a black node)
        assertz(node(Y)),      % potentially add it in Q
        fail.                  % backtrack to evaluate another neighbor of X
expand(_,Q,_):-
        collect(Q).            % we get here, end

collect(Q):-
        retract(node(X)),!,    % take X and if not under processing (not a grey one)
                               % as long as akb not empty
        insert_IL(X,Q),        % add in Q
        collect(Q).            % continue. How is possible with the SAME argument?
collect(Q).                    % end when akb empty
```

BFS – complete code

```
bfs(Q,R,R):- var(Q),!.
bfs([X|Q],Exp,R):-
        expand(X,Q,Exp),
        bfs(Q,[X|Exp],R).

expand(X,_,Exp):-
        is_edge(X,Y),
        not(member(Y, Exp)),
        assertz(node(Y)),
        fail.
expand(_,Q,_):-
        collect(Q).
```

```
collect(Q):-
        retract(node(X)), !,
        insert_IL(X,Q),
        collect(Q).
collect(Q).

insert_IL(X,[X|_]):-!.
insert_IL(X,[_|L]):-
        insert_IL(X,L).
```

```
[q] ?- bfs([a|_],[],Rez).
```

BFS – execution

```
[q] ?- bfs([a|_],[],Rez).        % for the first graph – search for a path
```

Step1

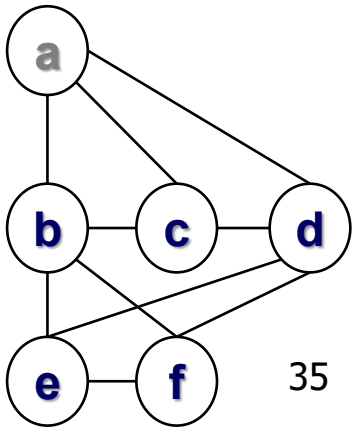
Exp: []

Q: [a|_]

```
bfs(Q,R,R):- var(Q),!.
bfs([X|Q],Exp,R):-              % [a|_]
        expand(X,Q,Exp),        % expand(a, …
        bfs(Q,[X|Exp],R).

expand(X,_,Exp):-
        is_edge(X,Y),
        not(member(Y, Exp)),
        assertz(node(Y)),
        fail.
expand(_,Q,_):-
        collect(Q).
```

```
edge(a,b).
edge(a,c).
edge(a,d).
edge(b,c).
edge(b,f).
edge(b,e).
edge(c,d).
edge(d,f).
edge(d,e).
edge(f,e).
```



BFS – execution

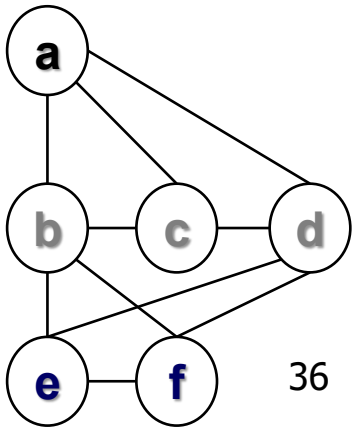
```
[q] ?- bf_search([a|_],[],Rez).
```
% for the first graph – search for a path

	Step1	Step2
Exp:	[]	[a]
Q:	[a|_]	[b,c,d|_]

```
bfs(Q,R,R):- var(Q),!.
bfs([X|Q],Exp,R):-          % [b| … ]
        expand(X,Q,Exp),    % expand(b, …
        bfs(Q,[X|Exp],R).

expand(X,_,Exp):-
        is_edge(X,Y),
        not(member(Y, Exp)),
        assertz(node(Y)),
        fail.
expand(_,Q,_):-
        collect(Q).
```

```
edge(a,b).
edge(a,c).
edge(a,d).
edge(b,c).
edge(b,f).
edge(b,e).
edge(c,d).
edge(d,f).
edge(d,e).
edge(f,e).
```



BFS – execution

```
[q] ?- bf_search([a|_],[],Rez).        % for the first graph – search for a path
```

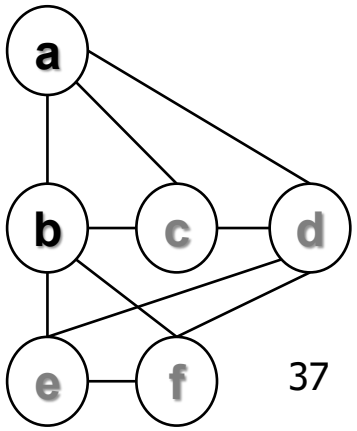
	Step1	Step2	Step3
Exp:	[]	[a]	[b,a]
Q:	[a|_]	[b,c,d|_]	[c,d,f,e|_]

% Why is `f` first?

```
bfs(Q,R,R):- var(Q),!.
bfs([X|Q],Exp,R):-
        expand(X,Q,Exp),
        bfs(Q,[X|Exp],R).

expand(X,_,Exp):-
        is_edge(X,Y),
        not(member(Y, Exp)),
        assertz(node(Y)),
        fail.
expand(_,Q,_):-
        collect(Q).
```

```
edge(a,b).
edge(a,c).
edge(a,d).
edge(b,c).
edge(b,f).
edge(b,e).
edge(c,d).
edge(d,f).
edge(d,e).
edge(f,e).
```



BFS – execution

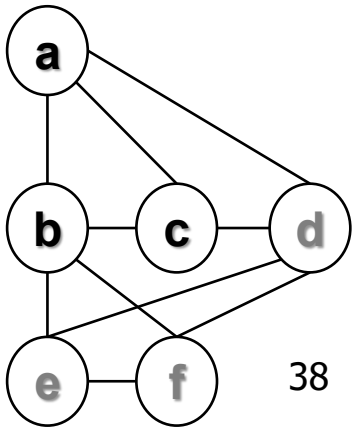
```
[q] ?- bf_search([a|_],[],Rez).
```
% for the first graph – search for a path

	Step1	Step2	Step3	Step4
Exp:	[]	[a]	[b,a]	[c,b,a]
Q:	[a|_]	[b,c,d|_]	[c,d,f,e|_]	[d,f,e|_]

```
bfs(Q,R,R):- var(Q),!.
bfs([X|Q],Exp,R):-
        expand(X,Q,Exp),
        bfs(Q,[X|Exp],R).

expand(X,_,Exp):-
        is_edge(X,Y),
        not(member(Y, Exp)),
        assertz(node(Y)),
        fail.
expand(_,Q,_):-
        collect(Q).
```

```
edge(a,b).
edge(a,c).
edge(a,d).
edge(b,c).
edge(b,f).
edge(b,e).
edge(c,d).
edge(d,f).
edge(d,e).
edge(f,e).
```



BFS – execution

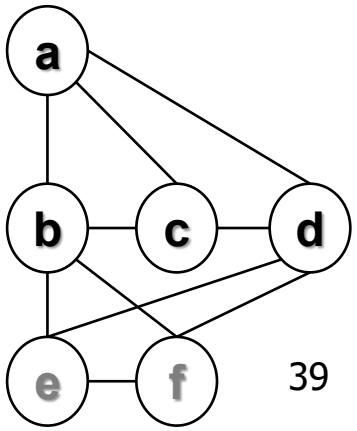
```
[q] ?- bf_search([a|_],[],Rez).        % for the first graph – search for a path
```

	Step1	Step2	Step3	Step4	Step5
Exp:	[]	[a]	[b,a]	[c,b,a]	[d,c,b,a]
Q:	[a|_]	[b,c,d|_]	[c,d,f,e|_]	[d,f,e|_]	[f,e|_]

```
bfs(Q,R,R):- var(Q),!.
bfs([X|Q],Exp,R):-
        expand(X,Q,Exp),
        bfs(Q,[X|Exp],R).

expand(X,_,Exp):-
        is_edge(X,Y),
        not(member(Y, Exp)),
        assertz(node(Y)),
        fail.
expand(_,Q,_):-
        collect(Q).
```

```
edge(a,b).
edge(a,c).
edge(a,d).
edge(b,c).
edge(b,f).
edge(b,e).
edge(c,d).
edge(d,f).
edge(d,e).
edge(f,e).
```



BFS – execution

```
[q] ?- bf_search([a|_],[],Rez).        % for the first graph – search for a path
```

	Step1	Step2	Step3	Step4	Step5	Step6	Step7
Exp :	[]	[a]	[b,a]	[c,b,a]	[d,c,b,a]	[f,d,c,b,a]	[e,f,d,c,b,a] DONE!
Q:	[a|_]	[b,c,d|_]	[c,d,f,e|_]	[d,f,e|_]	[f,e|_]	[e|_]	_

```
bfs(Q,R,R):- var(Q),!.
bfs([X|Q],Exp,R):-
        expand(X,Q,Exp),
        bfs(Q,[X|Exp],R).

expand(X,_,Exp):-
        is_edge(X,Y),
        not(member(Y, Exp)),
        assertz(node(Y)),
        fail.
expand(_,Q,_):-
        collect(Q).
```

```
edge(a,b).
edge(a,c).
edge(a,d).
edge(b,c).
edge(b,f).
edge(b,e).
edge(c,d).
edge(d,f).
edge(d,e).
edge(f,e).
```